Pocket Cart: Software Requirements Specification

Architecture, design tokens, components and implementation plan

Ashwamedha

23 October 2026 · Version 1.0 · Draft for build

1. Introduction

1.1 Purpose

This document describes what the Pocket Cart web app must do, how the frontend will be organised, which design rules to follow, and the order in which to build it. It is written so one developer can pick it up and build without guessing.

1.2 Product in one line

Pocket Cart is an Indian e-commerce storefront (tagline: *Shop more, Carry nothing*) with a shopping flow for customers and a dashboard for the store admin. Prices are in ₹.

1.3 Scope

In scope (Version 1): a frontend-only React app, responsive for desktop (1440 px design) and mobile (390 px design), using the free DummyJSON service for products, users, login and carts.

Out of scope (Version 1): a real backend, real payments, real order storage, email receipts, wishlist, coupons, product search results page design, and admin write operations that persist.

1.4 Source of truth

The Figma file *Pocket Cart – Wireframes & Prototype* (file key tYxpMHCg62PmXaFWwCGLHP). It has three pages: Cover, Components, and Wireframes & Prototype. The design is a **wireframe**, so colours and fonts are expected to be refined later. Everything visual is therefore kept in design tokens (Section 8).

1.5 Words used in this document

Term	Meaning
SRS	Software Requirements Specification, this document
Token	A named design value, such as a colour or spacing size
Service layer	The only code allowed to talk to an API
Mock	Fake data standing in for a real backend

2. Overall description

2.1 Where this fits

A browser app that talks to one external API (DummyJSON). Nothing else runs on a server for us. Later, the service layer can point to our own backend with no change to screens.

2.2 Who uses it

User	What they want
Guest	Browse, search, filter, view products, fill a cart
Customer (logged in)	Check out, see past orders, manage account
Admin	See sales numbers, top products and recent orders

2.3 Constraints

- Frontend only. No server code of our own.
- DummyJSON writes are **simulated**: adding to a cart or a product returns success but nothing is saved on their server. We therefore keep the cart and orders in the browser (localStorage).
- DummyJSON has no orders endpoint and no admin role. Orders and the admin numbers are built or derived on our side (see 6.2).
- Design covers 1440 px and 390 px. Tablet sizes must be handled by sensible scaling.

2.4 Assumptions

- The service meant by *DummyAPI* is **DummyJSON** (dummyjson.com), because it offers products, carts and login. The similarly named dummyapi.io has users and posts only, with no products. Please confirm.
- Login is optional until checkout.
- A single currency (₹) and a single language (English).

3. Screens and flows

3.1 Screen list

#	Screen	Desktop	Mobile	Notes
1	Home	Yes	Yes	Hero, categories, trending, deal of the day with countdown, recommended, value props
2	Product Listing	Yes	Yes	Filter sidebar (desktop) or filter sheet (mobile), sort, grid or list toggle, pagination or load more
3	Product Detail	Yes	Yes	Gallery, colour and warranty options, quantity, delivery pincode check, tabs, similar items
4	Cart	Yes	Item component only	Line items, order summary, delivery estimate, suggestions
5	Cart (empty)	Yes		Friendly empty state with a shop button
6	Checkout: Address	Yes		Step 2 of 4, saved addresses plus new address form
7	Checkout: Payment	Yes		Step 3 of 4, UPI, card, net banking, cash on delivery
8	Order Confirmation	Yes		Order number, items, address, total
9	Login / Sign up	Yes	Yes	Tabs, forgot password, social-style secondary button
10	Account: Orders	Yes		Sidebar menu, order cards, search orders
11	Admin Dashboard	Yes		Stats, revenue chart, top products, recent orders

The design shows a *Review* step (step 4 of checkout) in the step bar but has no screen for it. It is listed as an open question (Section 11).

3.2 Main customer journey

Home → Listing → Product Detail → Add to cart → Cart → Login (if needed) → Address → Payment → Order Confirmation → Account Orders.

4. Functional requirements

Priority: **M** = must have, **S** = should have, **C** = could have.

4.1 Authentication (AUTH)

ID	Requirement	Priority
AUTH-1	A user can log in with username and password using DummyJSON login	M
AUTH-2	The app stores the access token and loads the user profile after login	M
AUTH-3	Sign up shows a form and a success message, but is simulated (DummyJSON does not save new users)	S
AUTH-4	Log out clears the token and user data	M
AUTH-5	Protected pages (checkout, account) send guests to Login and return them afterwards	M
AUTH-6	Admin pages are available only to a user flagged as admin (mock rule, see 6.2)	S
AUTH-7	Field errors show under each input; a wrong password shows a clear message	M

4.2 Home (HOME)

ID	Requirement	Priority
HOME-1	Hero banner with sale text, a main button and a secondary button	M
HOME-2	Carousel dots that change the banner	C
HOME-3	Category tiles loaded from the categories API; each opens the listing filtered by that category	M
HOME-4	*Trending now* and *Recommended for you* rows, each showing product cards	M
HOME-5	Deal of the day with a live hours:minutes:seconds countdown	S
HOME-6	Value props strip: free shipping above ₹499, 7-day returns, secure payments, 24/7 support	M

4.3 Product listing (LIST)

ID	Requirement	Priority
LIST-1	Show products in a grid with a breadcrumb and the text *Showing 1–24 of N results*	M
LIST-2	Filter by category, price range, brand and rating	M
LIST-3	Sort by relevance, price low to high, price high to low, rating	M
LIST-4	Switch between grid and list view	C
LIST-5	Desktop uses numbered pagination; mobile uses a *Load more* button	M
LIST-6	Search bar in the navbar with a short typing delay (debounce)	M
LIST-7	Filters, sort and page live in the URL so a link can be shared	S
LIST-8	Loading skeletons, an empty result message and an error message with retry	M

4.4 Product detail (PDP)

ID	Requirement	Priority
PDP-1	Image gallery with a main image and four thumbnails	M
PDP-2	Show brand, title, rating, review count, price, old price and discount badge	M
PDP-3	Colour and warranty chips (the warranty add-on changes the price)	S
PDP-4	Quantity stepper, *Add to cart* and *Buy now*	M
PDP-5	Pincode box that shows a delivery date and shipping cost (rule-based mock)	S
PDP-6	Tabs for Description, Specifications and Reviews with the rating breakdown bars	M

ID	Requirement	Priority
PDP-7	*You may also like* row from the same category	M
PDP-8	A toast appears after adding to cart with a *View cart* link	S

4.5 Cart (CART)

ID	Requirement	Priority
CART-1	Add, remove and change the quantity of items	M
CART-2	The cart is kept after refresh (localStorage) for guests and customers	M
CART-3	Order summary shows subtotal, discount, shipping and total, all calculated in one helper	M
CART-4	Shipping is free above ₹499	M
CART-5	*Clear cart* asks for confirmation	S
CART-6	Empty cart shows the empty state and popular products	M
CART-7	The cart count shows on the navbar badge	M

4.6 Checkout (CHK)

ID	Requirement	Priority
CHK-1	Step bar shows Cart, Address, Payment and Review with done, current and upcoming states	M
CHK-2	Choose a saved address or add a new one (name, phone, address lines, city, pincode, state)	M
CHK-3	Choose a payment method; card form shows only when *Card* is chosen	M
CHK-4	Form checks: required fields, 6-digit pincode, 10-digit phone, card number format	M
CHK-5	*Place order* creates a mock order, clears the cart and opens the confirmation	M
CHK-6	Order summary stays visible on the right while checking out	M
CHK-7	No real card data is sent or stored; payment is simulated	M

4.7 Orders and account (ACC)

ID	Requirement	Priority
ACC-1	Account sidebar: Profile, Orders, Addresses, Wishlist, Payment methods, Log out	M
ACC-2	Orders page lists order cards with number, date, status badge, items and total	M
ACC-3	Search orders by name or number	S
ACC-4	Each order card has *View details* and *Buy again*	S
ACC-5	Profile and Addresses pages show and edit details stored in the browser	S
ACC-6	Wishlist and Payment methods show a *Coming soon* page in Version 1	C

4.8 Admin dashboard (ADM)

ID	Requirement	Priority
ADM-1	Sidebar: Dashboard, Products, Orders, Customers, Inventory, Coupons, Reports, Settings	M
ADM-2	Four stat cards (for example total revenue, orders, customers, products)	M
ADM-3	Revenue line chart for the last 30 days	M
ADM-4	Top products list with units sold	S
ADM-5	Recent orders table with order, customer, date, amount and status badge	M
ADM-6	Only Dashboard is built in Version 1; other menu items open a placeholder	M

5. Non-functional requirements

ID	Area	Requirement
NFR-1	Speed	Home loads and shows content within 3 seconds on a normal broadband connection

ID	Area	Requirement
NFR-2	Responsive	Works from 360 px to 1920 px wide; follows the 1440 and 390 designs
NFR-3	Accessibility	Keyboard friendly, visible focus, labelled inputs, alt text on images, text contrast of at least 4.5:1
NFR-4	Browsers	Latest Chrome, Edge, Firefox, Safari
NFR-5	Quality	No console errors; ESLint and Prettier pass
NFR-6	Resilience	Every API screen has loading, empty and error states
NFR-7	Security	No secrets in code; tokens never printed to the console; user input is never injected as raw HTML
NFR-8	Maintainability	Screens contain no direct API calls; colours and sizes come only from tokens
NFR-9	SEO basics	Page titles per route and meaningful headings

6. External interface: DummyJSON

6.1 Endpoint map

Base address: https://dummyjson.com

Need	Endpoint
Product list with paging and sorting	GET /products?limit=24&skip=0&sortBy=price&order=asc
One product	GET /products/{id}
Search	GET /products/search?q=phone
Category list	GET /products/categories
Products in a category	GET /products/category/{slug}
Login	POST /auth/login
Current user	GET /auth/me (Bearer token)
Carts of a user	GET /carts/user/{id}
Users (for admin tables)	GET /users

Test logins are listed at dummyjson.com/users. Please re-check the docs at dummyjson.com/docs before you code each call, since field names can change.

6.2 Things DummyJSON cannot do, and our answer

Gap	What we do
Cart changes are not saved on their server	Cart lives in a Zustand store persisted to localStorage
No orders endpoint	Orders are created by our order service and stored in localStorage; seed a few sample orders so the Orders page is never empty
No admin role	A fixed list of usernames counts as admin in the mock auth rule
No revenue history	Admin numbers are generated from seeded sample orders plus product and user counts
No address or payment API	Stored in the browser
Mixed filter support	Price, brand and rating filters run on our side on top of the returned page or the full list

6.3 Why this makes the backend upgrade easy

Every call above is wrapped in a service function with a fixed return shape (for example `getProducts({ page, category, sort })`). A real backend later means rewriting the inside of the services only. Screens, hooks and stores stay as they are.

7. Frontend architecture

7.1 Technology choices

Layer	Choice	Why
Framework	React 18 + Vite	Fast to start, matches your MERN experience
Language	JavaScript (ES6+), with JSDoc types	Keeps it simple; moving to TypeScript later is straightforward
Routing	React Router	Nested layouts and protected routes
Styling	CSS variables for tokens + CSS Modules (or Tailwind if you prefer)	Tokens work with either
Server data	TanStack Query	Caching, loading and error states with little code, and the same code works with a real backend
App state	Zustand (cart, auth, UI) with persist	Small and simple
Forms	React Hook Form + Zod	Clean validation for address, card and login
Charts	Recharts	Admin line chart
Dates and money	Intl API	No extra library needed
Testing	Vitest + React Testing Library	Cart maths and forms first

7.2 Folder structure

```
src/
├── app/                # router, providers, layouts, route guards
├── pages/              # thin screens that assemble features
├── features/
│   ├── auth/           # components, hooks, services, store
│   ├── products/       # listing, filters, detail, search
│   ├── cart/           # store, line item, summary, calculations
│   ├── checkout/       # steps, address form, payment form
│   ├── orders/         # order service, order card, orders page
│   └── admin/          # stat cards, chart, tables, derived data
├── components/
│   ├── ui/             # Button, Input, Checkbox, Radio, Badge, Rating...
│   ├── layout/         # Navbar, Footer, AppBar, TabBar, AdminSidebar
│   └── feedback/       # Toast, Skeleton, EmptyState, ErrorState
├── services/           # http client, token handling, error mapping
├── data/               # seed orders, mock admin data
├── hooks/              # useDebounce, useMediaQuery, useCountdown
├── styles/             # tokens.css, reset.css, global.css
├── utils/              # formatPrice, formatDate, calcTotals
└── constants/          # routes, shipping rules, admin usernames
```

A feature folder owns everything about that feature. Shared pieces move to `components/` only when a second feature needs them.

7.3 Layouts

Layout	Used by	Contains
PublicLayout	Home, Listing, Detail, Cart, Login	Navbar (App Bar on mobile), page area, Footer (Tab Bar on mobile)
CheckoutLayout	Address, Payment, Confirmation	Navbar, step bar, page area, Footer
AccountLayout	Orders and other account pages	Navbar, account sidebar, page area, Footer
AdminLayout	Dashboard and admin pages	Admin sidebar, top bar with search and avatar

Mobile does not get separate pages. The same page switches between desktop and mobile parts with CSS and a `useMediaQuery` hook only where the structure truly differs (filter sidebar vs filter sheet, pagination vs load more, navbar vs app bar).

7.4 Routes

Path	Screen	Guard
/	Home	None
/products	Listing (query: category, q, sort, page, price, brand, rating)	None
/products/:id	Product Detail	None
/cart	Cart	None
/login	Login and Sign up	Redirects away if already logged in
/checkout/address	Checkout step 2	Logged in, cart not empty
/checkout/payment	Checkout step 3	Logged in, address chosen
/order/confirmation/:id	Order Confirmation	Logged in
/account/orders	Orders	Logged in
/account/*	Profile, Addresses, others	Logged in
/admin	Dashboard	Admin only
*	Not found page	None

7.5 Data flow

```
Page → feature hook (TanStack Query) → feature service → http client → DummyJSON
                                    ↘ Zustand store (cart, auth, UI)
```

- **Server data** (products, categories, user) comes through TanStack Query hooks.
- **Client data** (cart, token, selected address, toast) lives in Zustand.
- **Rule:** components never call `fetch`. Services never import components.

7.6 State ownership

State	Owner	Saved in browser
Products, categories, product detail	TanStack Query cache	No
Cart items	Zustand cart store	Yes
Auth token and user	Zustand auth store	Yes
Checkout draft (address, payment choice)	Zustand checkout store	No (cleared on finish)
Orders (mock)	Order service	Yes
Filters and sort	URL query string	In the link
Toast and open sheets	Zustand UI store	No

7.7 Error and loading rules

- Each screen shows skeletons while loading, an empty message when there is no data, and an error box with *Try again* when a call fails.
- The http client turns every failure into one error shape: `{ status, message }`.
- A 401 response logs the user out and sends them to Login.

7.8 Path to a real backend

1. Keep service function names and return shapes fixed.
2. Put the base address in an environment variable (`VITE_API_URL`).
3. Replace localStorage order, address and cart logic with real endpoints inside the services.
4. Swap the mock admin rule for a role from the token.
5. Optional: move to TypeScript and generate types from the API.

8. Design tokens

Values below were read from the Figma component library. Items marked *estimated* come from layout sizes and should be checked against the final visual design.

8.1 Colour

Token	Value	Used for
–color-primary	#1B2A4A	Primary button, headings, active page, secondary button border and text
–color-accent	#F28C28	CTA button, toast link
–color-text	#111827	Body text, toast background
–color-text-muted	#6B7280	Labels, breadcrumb, helper text
–color-border	#D1D5DB	Card, input and pagination borders
–color-surface-alt	#E5E7EB	Image placeholders
–color-surface-soft	#F3F4F6	Badge background
–color-surface	#FFFFFF	Cards, page, secondary button
–color-star	#F59E0B	Rating stars

Status colours (success, warning, error) are not defined in the wireframe. Add them when the visual design is ready; badges currently use the soft grey.

8.2 Typography

Font family: **Inter** (weights 400, 500, 600, 700), with system fonts as fallback.

Token	Size	Weight	Used for
–text-badge	11 px	500	Badges
–text-caption	12 px	400 / 500	Labels, rating text, stat captions
–text-small	13 px	400 / 500	Breadcrumb, pagination
–text-body	14 px	400 / 500 / 600	Body text, buttons, toast
–text-section	20 px	700	Section titles
–text-stat	28 px	700	Stat card numbers
–text-page-title	about 32 px (estimated)	700	Page titles such as Shopping cart
–text-hero	about 42 px (estimated)	700	Hero heading

8.3 Spacing, shape and layout

Token	Value	Notes
–space-1 to –space-6	4, 8, 12, 16, 24, 32 px	Gaps seen in components are 4, 8, 16 and 32
–page-margin	64 px desktop, 16 px mobile	Side margin of every page
–container-max	1312 px	Content width inside a 1440 px page
–radius-button	6 px	Buttons, pagination boxes
–radius-card	8 px	Cards, toast, stat cards
–radius-badge	10 px	Badges
–button-padding	12 px 20 px	All buttons
–sidebar-width	260 px	Filter and account sidebars
–admin-sidebar-width	240 px	Admin menu
–product-card-width	260 px desktop, 171 px mobile	Product card
–breakpoint-mobile	390 px design, switch below 768 px	Mobile layout
–breakpoint-desktop	1440 px design, switch at 1024 px and above	Desktop layout

8.4 Starter file: styles/tokens.css

```
:root {
  --color-primary: #1b2a4a;
  --color-accent: #f28c28;
  --color-text: #111827;
  --color-text-muted: #6b7280;
  --color-border: #d1d5db;
  --color-surface: #ffffff;
  --color-surface-soft: #f3f4f6;
  --color-surface-alt: #e5e7eb;
  --color-star: #f59e0b;

  --font-family: "Inter", system-ui, -apple-system, "Segoe UI", sans-serif;
  --text-badge: 11px;
  --text-caption: 12px;
  --text-small: 13px;
  --text-body: 14px;
  --text-section: 20px;
  --text-stat: 28px;

  --space-1: 4px;
  --space-2: 8px;
  --space-3: 12px;
  --space-4: 16px;
  --space-5: 24px;
  --space-6: 32px;

  --radius-button: 6px;
  --radius-card: 8px;
  --radius-badge: 10px;

  --page-margin: 16px;
  --container-max: 1312px;
}

@media (min-width: 1024px) {
```

```
  :root { --page-margin: 64px; }
}
```

9. Component inventory

9.1 Base components (components/ui)

Component	Variants and states	Main props
Button	primary (navy), secondary (outline), cta (orange); hover, disabled, loading	variant, onClick, disabled, loading, fullWidth
Input	default, focus, error, disabled	label, type, error, register
SearchBar	default, with suggestions later	value, onChange, placeholder
Checkbox	checked, unchecked	label, checked, onChange
RadioOption	selected, unselected (a card-style row with title and subtext)	title, description, selected, onSelect
QuantityStepper	min 1, max stock	value, onChange, min, max
Badge	neutral now; success, warning, error later	label, tone
Rating	star row plus score and count	value, count
Breadcrumb	path list	items
Pagination	previous, numbers, ellipsis, next	page, totalPages, onChange
Toast	message with action link	message, actionLabel, onAction
SectionHeader	title plus *View all* link	title, linkTo

9.2 Commerce components (features/*)

Component	Used on	Main props
ProductCard	Home, Listing, Detail, Cart	product, onAdd
ProductGrid	Home, Listing	products, view (grid or list)
CategoryTile	Home	category
FilterSidebar (desktop) / FilterSheet (mobile)	Listing	filters, onChange, onClear
ProductGallery	Detail	images
OptionChips	Detail (colour, warranty)	options, value, onChange
DeliveryChecker	Detail	pincode, onCheck
ReviewSummary	Detail	rating, breakdown
CartLineItem	Cart	item, onQtyChange, onRemove
OrderSummary	Cart, Checkout	totals
CheckoutSteps	Checkout	current
AddressForm, AddressOption	Checkout, Account	values, onSubmit
PaymentOption, CardForm	Checkout	method, onChange
OrderCard	Account	order
StatCard	Admin	label, value, trend
RevenueChart	Admin	data
DataTable	Admin	columns, rows

9.3 Layout and feedback components

Component	Notes
Navbar	Logo, search, account, cart with count badge

Component	Notes
Footer	Link columns
AppBar, BackBar, TabBar, StickyBar	Mobile only (App Bar on Home, Back Bar on inner pages, Tab Bar at bottom, Sticky Bar with two buttons on Product Detail)
AdminSidebar, AdminTopBar	Admin layout
HeroBanner, DealBanner, ValueProps	Home sections
Skeleton, EmptyState, ErrorState	Loading, empty and error views

10. Implementation order

Each phase ends with something you can open in the browser. Do not start a phase until the one before it works.

Phase	Goal	What to build	Done when
0	Project setup	Vite app, folders, ESLint, Prettier, tokens.css, Inter font, router with empty pages, env file	App runs and every route shows a placeholder
1	Base UI	Button, Input, Checkbox, RadioOption, Badge, Rating, QuantityStepper, Breadcrumb, Pagination, Toast, SectionHeader	A temporary *kitchen sink* page shows all components in every state
2	Layouts	Navbar, Footer, PublicLayout, mobile AppBar and TabBar	All pages share the frame; it adapts on mobile
3	Data layer	http client, product and category services, TanStack Query hooks, Skeleton, EmptyState, ErrorState	Console test shows products from DummyJSON
4	Home	Hero, CategoryTile, ProductCard, product rows, DealBanner with countdown, ValueProps	Home matches the design on both sizes
5	Listing	ProductGrid, FilterSidebar and FilterSheet, sort, search, pagination, URL query sync	Filters and pages work and survive a refresh
6	Product Detail	Gallery, options, stepper, delivery checker, tabs, similar items	Any product ID opens a full page
7	Cart	Cart store with persistence, CartLineItem, OrderSummary, empty state, navbar count, toast	Add, change and remove work; totals are correct
8	Auth	Auth store, login and sign up forms, route guards, redirect back after login	Test user can log in and out; guards work
9	Checkout	CheckoutLayout, steps, address form, payment options, validation, order service, Confirmation page	A full order can be placed end to end
10	Account	AccountLayout, Orders page with search, Profile and Addresses	Placed orders show up in the list
11	Admin	AdminLayout, StatCards, RevenueChart, top products, recent orders table, admin guard	Dashboard renders with seeded data
12	Quality	Accessibility pass, tests for calculations and forms, performance check, empty and error screens, README	All must-have requirements pass; Lighthouse accessibility 90+
13	Release	Build, deploy (Vercel or Netlify),	Public link works

Phase	Goal	What to build	Done when
		project screenshots and demo link	

Suggested pace: phases 0 to 3 are the foundation (about one week); 4 to 7 deliver the shopping experience (about two weeks); 8 to 10 complete the customer journey (about one to two weeks); 11 to 13 finish the project. These are rough guides only.

10.1 Test checklist for cart and checkout

- Totals are right with one item, many items and quantity changes.
- Shipping is free at ₹499 and above and charged below it.
- The cart survives a page refresh.
- A guest who clicks Checkout is sent to Login and then returned.
- Invalid pincode, phone and card entries show errors and block the next step.

11. Risks and open questions

#	Item	Suggested handling
1	DummyJSON may be slow or unavailable	Cache with TanStack Query; keep a small local products file as backup
2	The API and the design disagree on fields (for example colour options, warranty, brand)	Map them in a single `normalizeProduct` function and fill missing options with defaults
3	The design is a wireframe, so colours and fonts may change	Use tokens only; never type a colour or size in a component
4	The Review step (step 4) has no screen	Decision needed: build a Review screen or merge it into Payment
5	Menu links without designs (Wishlist, Payment methods, Coupons, Reports and others)	Show *Coming soon* pages
6	Image placeholders in the design	Use product thumbnails and images from the API
7	Which service is *DummyAPI*?	Confirm DummyJSON before Phase 3

12. Approval

This is a personal portfolio project. The author approves the document by starting Phase 0.